

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
```

```
data = {
    "Area": [1200, 1500, 1800, 2000, 1000, 2200, 1700, 2500, 1300, 1900,
             1600, 2300, 1100, 2100, 1450, 2700, 1550, 1950, 1250, 2400],

    "Bedrooms": [2, 3, 3, 4, 2, 4, 3, 4, 2, 3,
                 3, 4, 2, 4, 3, 5, 3, 4, 2, 4],

    "Age": [10, 8, 5, 6, 15, 4, 7, 3, 12, 5,
            9, 2, 14, 5, 10, 2, 8, 6, 11, 3],

    "Distance": [5, 4, 3, 2, 7, 3, 5, 2, 6, 4,
                 5, 3, 8, 2, 6, 1, 4, 3, 7, 2],

    "Price": [2500000, 3200000, 4000000, 4500000, 2100000,
              5000000, 3500000, 5800000, 2800000, 4200000,
              3400000, 5300000, 2300000, 4800000, 3000000,
              6500000, 3300000, 4400000, 2600000, 5500000]
}

df = pd.DataFrame(data)

df
```

	Area	Bedrooms	Age	Distance	Price	
0	1200	2	10	5	2500000	
1	1500	3	8	4	3200000	
2	1800	3	5	3	4000000	
3	2000	4	6	2	4500000	
4	1000	2	15	7	2100000	
5	2200	4	4	3	5000000	
6	1700	3	7	5	3500000	
7	2500	4	3	2	5800000	
8	1300	2	12	6	2800000	
9	1900	3	5	4	4200000	
10	1600	3	9	5	3400000	
11	2300	4	2	3	5300000	
12	1100	2	14	8	2300000	
13	2100	4	5	2	4800000	
14	1450	3	10	6	3000000	
15	2700	5	2	1	6500000	
16	1550	3	8	4	3300000	
17	1950	4	6	3	4400000	
18	1250	2	11	7	2600000	
19	2400	4	3	2	5500000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print("Dataset Shape:", df.shape)

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
```

```
print(df.describe())
```

Dataset Shape: (20, 5)

Dataset Information:
 <class 'pandas.core.frame.DataFrame'>
 RangeIndex: 20 entries, 0 to 19
 Data columns (total 5 columns):
 # Column Non-Null Count Dtype
 --- ---
 0 Area 20 non-null int64
 1 Bedrooms 20 non-null int64
 2 Age 20 non-null int64
 3 Distance 20 non-null int64
 4 Price 20 non-null int64
 dtypes: int64(5)
 memory usage: 932.0 bytes
 None

Missing Values:

Area 0
 Bedrooms 0
 Age 0
 Distance 0
 Price 0
 dtype: int64

Statistical Summary:

	Area	Bedrooms	Age	Distance	Price
count	20.000000	20.000000	20.000000	20.000000	2.000000e+01
mean	1775.000000	3.200000	7.250000	4.100000	3.935000e+06
std	491.372942	0.894427	3.850837	1.97084	1.263360e+06
min	1000.000000	2.000000	2.000000	1.000000	2.100000e+06
25%	1412.500000	2.750000	4.750000	2.750000	2.950000e+06
50%	1750.000000	3.000000	6.500000	4.000000	3.750000e+06
75%	2125.000000	4.000000	10.000000	5.250000	4.850000e+06
max	2700.000000	5.000000	15.000000	8.000000	6.500000e+06

```
X = df[["Area", "Bedrooms", "Age", "Distance"]]
```

```
y = df["Price"]
```

```
print("Features:")
```

```
print(X.head())
```

```
print("\nTarget:")
```

```
print(y.head())
```

Features:

	Area	Bedrooms	Age	Distance
0	1200	2	10	5
1	1500	3	8	4
2	1800	3	5	3
3	2000	4	6	2
4	1000	2	15	7

Target:

0	2500000
1	3200000
2	4000000
3	4500000
4	2100000

Name: Price, dtype: int64

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training Data Shape:", X_train.shape)
```

```
print("Testing Data Shape:", X_test.shape)
```

Training Data Shape: (16, 4)

Testing Data Shape: (4, 4)

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
print("Scaled Training Data:")
```

```
print(X_train_scaled[:5])
```

```
Scaled Training Data:
[[-1.01287819 -1.44115338  1.16848619  0.86116626]
 [ 0.97153623  1.12089708 -0.88036631 -0.66979598]
 [ 1.19202672  1.12089708 -1.39257943 -0.66979598]
 [ 0.53055524  1.12089708 -0.36815318 -1.18011672]
 [-1.12312344 -1.44115338  0.91237963  1.371487  ]]
```

```
class LinearRegressionGD:

    def __init__(self, learning_rate=0.01, iterations=1000):

        self.learning_rate = learning_rate
        self.iterations = iterations

        self.weights = None
        self.bias = None

        self.cost_history = []

    def fit(self, X, y):

        n_samples, n_features = X.shape

        # Initialize weights and bias
        self.weights = np.zeros(n_features)
        self.bias = 0

        # Gradient Descent
        for i in range(self.iterations):

            # Step 1: Make prediction
            y_predicted = np.dot(X, self.weights) + self.bias

            # Step 2: Calculate error
            error = y_predicted - y

            # Step 3: Calculate gradients
            dw = (1 / n_samples) * np.dot(X.T, error)

            db = (1 / n_samples) * np.sum(error)

            # Step 4: Update weights and bias
            self.weights = self.weights - self.learning_rate * dw

            self.bias = self.bias - self.learning_rate * db

            # Step 5: Calculate Mean Squared Error
            cost = np.mean(error ** 2)

            self.cost_history.append(cost)

    def predict(self, X):

        return np.dot(X, self.weights) + self.bias
```

```
model = LinearRegressionGD(
    learning_rate=0.01,
    iterations=1000
)

model.fit(
    X_train_scaled,
    y_train.values
)

print("Model Training Completed!")
```

Model Training Completed!

```
y_pred = model.predict(X_test_scaled)

print("Predicted Prices:")
print(y_pred)
```

```
Predicted Prices:
[2686260.94662165 4506349.05405855 6095953.27865481 3498796.32605071]
```

```
mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Model Performance")
print("-----")

print("Mean Squared Error:", mse)

print("Root Mean Squared Error:", rmse)

print("R2 Score:", r2)
```

```
Model Performance
-----
Mean Squared Error: 74634064756.68619
Root Mean Squared Error: 273192.35852542834
R2 Score: 0.9675856396279322
```

```
results = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

results
```

	Actual Price	Predicted Price	
0	2500000	2.686261e+06	
1	4400000	4.506349e+06	
2	6500000	6.095953e+06	
3	3200000	3.498796e+06	

Next steps: [Generate code with results](#) [New interactive sheet](#)

```
plt.figure(figsize=(10, 6))

plt.plot(model.cost_history)

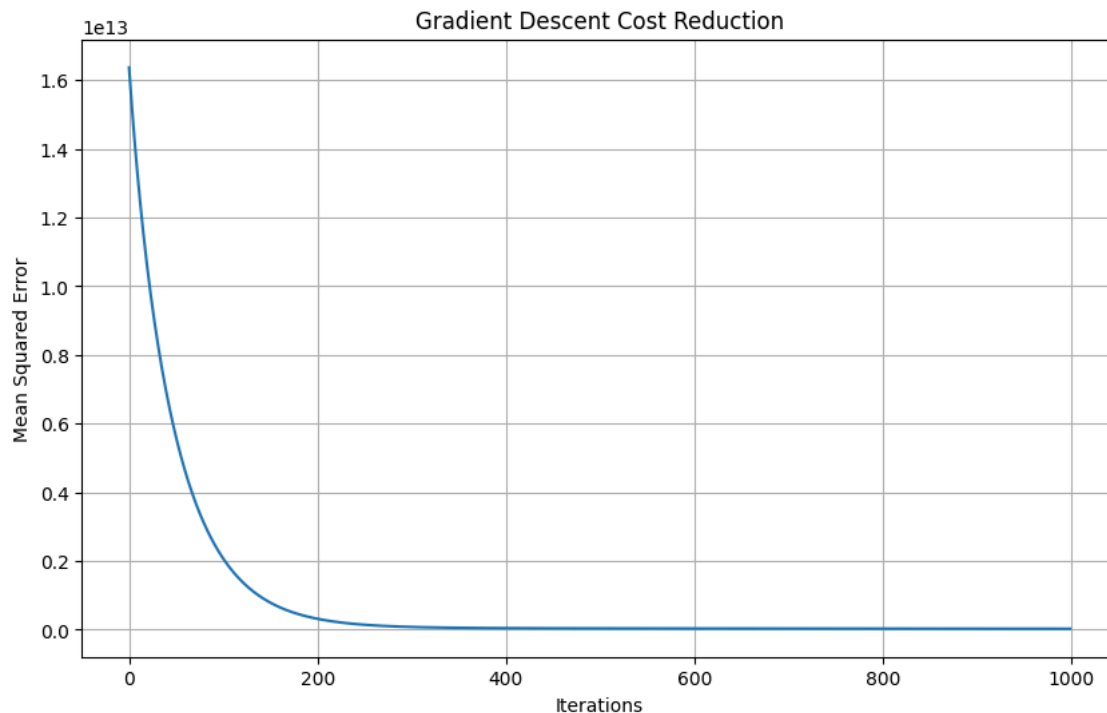
plt.xlabel("Iterations")

plt.ylabel("Mean Squared Error")

plt.title("Gradient Descent Cost Reduction")

plt.grid()

plt.show()
```



```
plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Prices")

plt.grid()

plt.show()
```



```
new_house = np.array([
    [1800, 3, 5, 3]
])

new_house_scaled = scaler.transform(new_house)

predicted_price = model.predict(new_house_scaled)
```

```
print("Predicted House Price:", predicted_price[0])
```

```
Predicted House Price: 4145069.6175259952
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
warnings.warn(
```

```
new_house = np.array([
    [2500, 4, 2, 2]
])
```

```
learning_rates = [0.001, 0.005, 0.01, 0.05]

results_lr = []

for lr in learning_rates:

    temp_model = LinearRegressionGD(
        learning_rate=lr,
        iterations=1000
    )

    temp_model.fit(
        X_train_scaled,
        y_train.values
    )

    temp_predictions = temp_model.predict(
        X_test_scaled
    )

    temp_mse = mean_squared_error(
        y_test,
        temp_predictions
    )

    temp_rmse = np.sqrt(temp_mse)

    temp_r2 = r2_score(
        y_test,
        temp_predictions
    )

    results_lr.append([
        lr,
        temp_rmse,
        temp_r2
    ])

comparison_df = pd.DataFrame(
    results_lr,
    columns=[
        "Learning Rate",
        "RMSE",
        "R2 Score"
    ]
)

comparison_df
```

	Learning Rate	RMSE	R2 Score	
0	0.001	1.357197e+06	0.200008	
1	0.005	3.118893e+05	0.957752	
2	0.010	2.731924e+05	0.967586	
3	0.050	1.390206e+05	0.991606	

Next steps: [Generate code with comparison_df](#) [New interactive sheet](#)

```
sk_model = LinearRegression()

sk_model.fit(
    X_train_scaled,
    y_train
)

sk_predictions = sk_model.predict(
```

```

    X_test_scaled
)

sk_mse = mean_squared_error(
    y_test,
    sk_predictions
)

sk_rmse = np.sqrt(sk_mse)

sk_r2 = r2_score(
    y_test,
    sk_predictions
)

print("Scikit-learn Linear Regression Results")

print("MSE:", sk_mse)

print("RMSE:", sk_rmse)

print("R2 Score:", sk_r2)

```

Scikit-learn Linear Regression Results
MSE: 12129542236.80599
RMSE: 110134.20103131447
R2 Score: 0.9947320120578476

```

final_comparison = pd.DataFrame({
    "Model": [
        "Gradient Descent",
        "Scikit-learn Linear Regression"
    ],
    "RMSE": [
        rmse,
        sk_rmse
    ],
    "R2 Score": [
        r2,
        sk_r2
    ]
})

final_comparison

```

	Model	RMSE	R2 Score	
0	Gradient Descent	273192.358525	0.967586	
1	Scikit-learn Linear Regression	110134.201031	0.994732	

Next steps:

[Generate code with final_comparison](#)[New interactive sheet](#)

```

final_comparison = pd.DataFrame({
    "Model": [
        "Gradient Descent",
        "Scikit-learn Linear Regression"
    ],
    "RMSE": [
        rmse,
        sk_rmse
    ],
    "R2 Score": [
        r2,
        sk_r2
    ]
})

final_comparison

```

	Model	RMSE	R2 Score	
0	Gradient Descent	273192.358525	0.967586	